```sql
CREATE TABLE categories (
    category_id INT PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category_id INT,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    order_status VARCHAR(20) NOT NULL,
    order_date DATE NOT NULL,
    FOREIGN KEY (category_id)
        REFERENCES categories(category_id)
);

INSERT INTO categories VALUES
(1, 'Electronics'),
(2, 'Clothing'),
(3, 'Home'),
(4, 'Books');

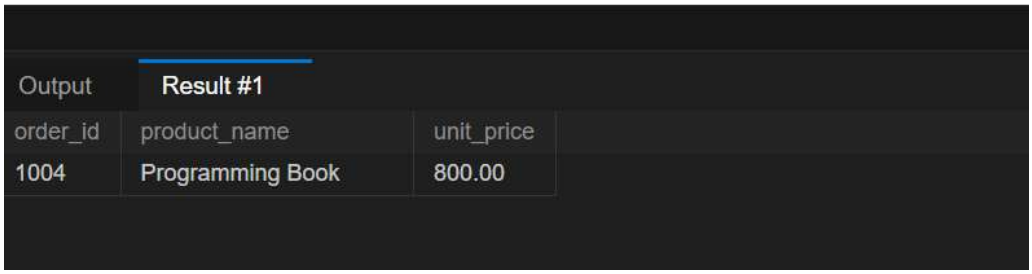
INSERT INTO orders VALUES
(1001, 'Wireless Headphones', 1, 2, 2500.00, 'Delivered', '2026-01-05'),
(1002, 'T-Shirt', 2, 3, 500.00, 'Delivered', '2026-01-07'),
(1003, 'Office Chair', 3, 1, 4500.00, 'Pending', '2026-01-10'),
(1004, 'Programming Book', 4, 2, 800.00, 'Delivered', '2026-01-12'),
(1005, 'Smart Watch', 1, 1, 3500.00, 'Cancelled', '2026-01-15'),
(1006, 'Jeans', 2, 2, 1200.00, 'Delivered', '2026-01-18'),
(1007, 'Table Lamp', 3, 2, 1500.00, 'Pending', '2026-01-20'),
(1008, 'Laptop', 1, 1, 55000.00, 'Delivered', '2026-01-22'),
(1009, 'Novel', 4, 3, 400.00, 'Delivered', '2026-01-25'),
(1010, 'Jacket', 2, 1, 2200.00, 'Cancelled', '2026-01-28');
```

Query 1 —Pattern-based filtering

```sql

    SELECT order_id, product_name, unit_price
    FROM orders
    WHERE product_name LIKE '%Book%'
    ORDER BY unit_price DESC
    LIMIT 5;

```



Output | Result #1

order_id	product_name	unit_price
1004	Programming Book	800.00

Purpose: To search for products whose names contain a specific text pattern, sort them by unit price in descending order, and display only the required number of records.

Query 2 — Range-based filtering

```
9   • SELECT order_id, product_name, order_date, order_status
10    FROM orders
11    WHERE order_date BETWEEN '2026-01-10' AND '2026-01-25'
12    ORDER BY order_date ASC;
13
```

Output | Result #2

order_id	product_name	order_date	order_status
1003	Office Chair	01/10/2026	Pending
1004	Programming Book	01/12/2026	Delivered
1005	Smart Watch	01/15/2026	Cancelled
1006	Jeans	01/18/2026	Delivered
1007	Table Lamp	01/20/2026	Pending
1008	Laptop	01/22/2026	Delivered
1009	Novel	01/25/2026	Delivered

Purpose: To retrieve orders placed within a specified date range and arrange them chronologically.

Query 3 — Combine related tables

```sql
SELECT
    c.category_id,
    c.category_name,
    o.order_id,
    o.product_name,
    o.order_status
FROM categories c
LEFT JOIN orders o
    ON c.category_id = o.category_id
ORDER BY c.category_name, o.order_id;
```

Output | Result #3

category_id	category_name	order_id	product_name	order_status
4	Books	1004	Programming Book	Delivered
4	Books	1009	Novel	Delivered
2	Clothing	1002	T-Shirt	Delivered
2	Clothing	1006	Jeans	Delivered
2	Clothing	1010	Jacket	Cancelled
1	Electronics	1001	Wireless Headphones	Delivered
1	Electronics	1005	Smart Watch	Cancelled
1	Electronics	1008	Laptop	Delivered
3	Home	1003	Office Chair	Pending
3	Home	1007	Table Lamp	Pending

Purpose: To display all product categories along with their associated orders, including categories that have no matching orders.

Query 4 — Rounded average analysis

```
27
28  SELECT
29      c.category_name,
30      ROUND(AVG(o.quantity * o.unit_price), 2)
31          AS average_order_value
32  FROM categories c
33  LEFT JOIN orders o
34      ON c.category_id = o.category_id
35  GROUP BY c.category_id, c.category_name
36  ORDER BY average_order_value DESC;
37
38
```

Output | Result #4

category_name	average_order_value
Electronics	21166.67
Home	3750.00
Clothing	2033.33
Books	1400.00

Purpose: To calculate the average order value for each product category, round the results to two decimal places, and rank categories by their average order value.